

Parametrized and Exact Algorithms

Notes by Arjun Maneesh Agarwal

based on course by G. Philip

The main book will be Parametrized Algorithms by Cygan et. al (chapters 1 to 7, 13 and 15). Other than that < copy from moodle >.

Table of Contents

1. Philip's Wisdom	1
2. Brief Intro and Motivation	1
3. Definitions	3
4. Exact Exponential Algorithms	4
5. Kernalization	4
5.1. Planar Independent Set	6
6. Feedback Vertex Set	7
7. Feedback Arc Set	7

1. Philip's Wisdom

“Why do you want to solve an algorithms problem, probably to make more money. Or kill more people. Or maybe save more people.”

2. Brief Intro and Motivation

Parametrized Algorithms starts where the first algorithms course ends: NP Completeness.

Most algorithms we saw in the algorithms course have complexity $O(n^c)$ for some constant c . This raises the question if that is true of all problems, sadly, this is where NP Hardness and Completeness comes in. While we don't know if we can solve them polytime, these NP Complete problems span thousands of fields and experts in their field have tried finding fast algorithms. As these problems are complete, all reduce to each other which implies that if one of them succeed, all of them succeed. Unfortunately, they have all failed, which is the sort of silence that speaks quite loudly.

So how do we cope with this? One is Approx Algorithms. The other idea is Random Algorithms. Another is heuristics and meta-heuristics (taken from physics and biology and what not).

In theory, NP Completeness implies all the problems are equally hard modulo some polynomial time translations. However, some problems were easier to solve in practice.

Example

Type Inference for Hindley-Milner type system is unconditionally not solvable in polynomial time (in the size n of the input programme) as it is hard for the class EXP (that is problems that can be solved in $O(2^{\text{poly}})$ time) (from Maivson, POPL 1990) and $P \subsetneq \text{EXP}$ (Time Hierarchy theorem)

However, we don't think about the size of programme while writing code. And the compiler happily (or sadly) infers the type quite quickly.

Theorem 2.1. *Type inference for Hindley-Milner Type system can be solved in $O(2^k n)$ time where*

- n is the size of input programme,
- k is the nesting depth of types in the programmes

(Lichtenstein and Porveli, POPL 1985)

The idea is to pick something that tends to something that is usually small in practice and limit the exponential blow up to that parameter.

So our roadmap is: pick a parameter k that could be natural or something more convoluted, try to find an algorithm of time complexity $O(f(k)n^c)$. If we succeed the problem is Fixed Parameter Tractable aka FPT.

Definition: Vertex Cover

Vertex cover of a graph G is a subset $S \subseteq V(G)$ such that $G \setminus S$ has no edges.

Definition: Vertex Cover (Standard Parametrization)

Input: Graph G , $k \in \mathbb{N}$

Parameter: k

Question: Does G have a vertex cover of size $\leq k$?

Remark

This can be thought of as choosing a subset as guard posts such that all the roads are guarded.

Also, please make sure to not confuse this with **Dominating Set** (which is choosing vertices to put 1 edge radius umbrellas to cover everyone).

While vertex cover is NP Complete, we can see a simple algorithm that runs in $\mathcal{O}^*(2^k)$ time (we can also do it in $\mathcal{O}^*(1.4656^k)$ time as we'll later see and in $\mathcal{O}^*(1.2529^k)$ as we shall not see).

The idea is that given an edge, one of the two vertices neighboring it have to be in the vertex cover. This is a local property and can be abused to get the algorithm.

VERTEX COVER IN $O(2^k n)$ TIME

```

1 func existVertexCover( $G, k$ ):
2   |   pick edge  $uv \in E(G)$ 
3   |   return existVertexCover( $G \setminus \{u\}, k - 1$ ) OR existVertexCover( $G \setminus \{v\}, k - 1$ )

```

3. Definitions

Definition: Classical Language

Let Σ be a finite alphabet, for example $\Sigma = \{0, 1\}$ or $\Sigma = \text{ASCII}$. A classical language over Σ is $L \subseteq \Sigma^*$, for example $L = \{\text{All valid Haskell programmes}\}$.

Definition: Parametrized Language

A parametrized language over Σ is $L \subseteq \Sigma^* \times \mathbb{N}$, for example $L = \{(x, k) \mid x \text{ is a valid Haskell programme with } k \text{ lines}\}$.
For a fixed k , $L_k = \{(x, k) \mid (x, k) \in L\}$ is the k -th slice of L .

Definition: Slice-wise Polynomial Time (XP)

A parametrized language L is slice-wise polynomial time solvable (XP) if there exists an algorithm $\mathcal{A}$ and a computable function f such that:

- For all x, k , $\mathcal{A}$ runs in $\leq |x|^{f(k)}$ time on input (x, k)
- $(x, k) \in L \iff \mathcal{A}((x, k)) = \text{YES}$

Definition: Fixed Parameter Tractable (FPT)

A parametrized language L is fixed parameter tractable (FPT) if there exists an algorithm $\mathcal{A}$ and a computable function f such that:

- For all x, k , $\mathcal{A}$ runs in $\leq f(k) |x|^c$ time on input (x, k) where c is a constant independent of $|x|$ and k .
- $(x, k) \in L \iff \mathcal{A}((x, k)) = \text{YES}$.

For notational convenience, $f(k) |x|^c$ is abbreviated $\mathcal{O}^*(f(k))$.

One motivation could be this table:

$\frac{n^{k+1}}{2^k n}$	$n = 50$	$n = 100$	$n = 150$
$k = 2$	625	2500	5625
$k = 3$	15625	125000	421875

$k = 10$	10^{12}	8×10^{13}	3.7×10^{16}
$k = 20$	1.8×10^{26}	5×10^{30}	2.1×10^{35}

4. Exact Exponential Algorithms

An example could be Hamiltonian Path.

Definition: Hamiltonian Path

Input: Given a graph G and vertex $v \in V$

Question: Does there exist a path P starting at v which covers all vertices of G without repeating vertices or edges.

This problem is NP Hard, however using a DP on sets, we can do this in $\mathcal{O}^*(2^n)$ instead of the brute force $\mathcal{O}^*(n!)$.

Another example is Chromatic Number

Definition: Chromatic Number

Input: Given a graph G and $k \in \mathbb{N}$

Question: Can we properly color the graph G using k colors

While there is a $\mathcal{O}^*(2^k)$ algorithm, we can see an interesting and simple $\mathcal{O}^*(3^k)$ algorithm.

Consider all the subset of vertices and store all the independent sets (that means can be 1 colored). Now consider subsets of vertices and store all the ones that can be partitioned into independent sets (which will take 3^n time as we are basically 3 partitioning the vertices into $S = X \sqcup Y$ and $V \setminus S$) giving us the 2-colorable subsets. Repeat with 3 coloring and so on. This will give an $\underbrace{\mathcal{O}^*(2^n) + \mathcal{O}^*(3^n) + \mathcal{O}^*(3^n) + \dots + \mathcal{O}^*(3^n)}_{k \text{ terms}} = \mathcal{O}^*(3^n)$.

5. Kernalization

TODO. *Introduction with sorting and cycle detection idea*

Example : European Rail Network Problem

Given an input of 1, 40, 000 trains, 25, 000 stations and 16, 00, 000 single train stops (valid trains and station pairs).

Our question is if there is any subset T of trains, they wanted to find a smallest set S of stations such that each train T stops at atleast one of the stations in S .

This is roughly equivalent to the red-blue dominating set which is NP Hard. However, Karsten Werhe gave a set of reduction rules (similar to the cycle detection) that would split this into small cases which he could solve by hand.

TODO. *Section 4.6 of Fundamentals of Parametrized Complexity by Downey and Fellows*

This is a weird place as if we had a polynomial algorithm that could reduce the size of an NP problem by even 1 bit, we could just use the same algorithm to keep reducing it to 1 single bit (which would be the answer).

So what do we do?

TODO. *Parametrization of Kernel*

For vertex cover, we can use the following reduction rules:

- If there is a vertex v of degree zero in G , then delete v from G to get G' . Output: (G', k)
- If there is a vertex v of degree $\geq k + 1$ in G , then delete v from G to get G' . Output: $(G', k - 1)$

Let $(\hat{G}, \hat{k})$ be an instance to which none of the reduction rules applies. We then brute force from there.

We now need to prove:

1. Yes $\iff$ Yes
2. $|\hat{G}| + \hat{k} \leq f(k)$ for some computable function of k .

Proof.

TODO.

■

Definition: Kernalization

Let $L \subseteq \Sigma^* \times \mathbb{N}$ be a parametrized language. A **kernalization** algorithm (also called **reduction to problem kernel**) for L is an algorithm that given an instance (x, k) runs in time polynomial in $(|x| + k)$ and outputs an instance (x', k') such that

1. $|x'| + k' \leq g(k)$ for some computable g
2. $(x, k) \in L \iff (x', k') \in L$

(x', k') is called a kernel of L of size $g(k)$.

First Theorem of Parametrized Complexity 5.1. *A computable parametrized language L has an FPT algorithm if and only if it has a kernalization algorithm.*

Proof. Let there be a $O(f(x))$ (not necessarily polynomial) algorithm for the decision problem where x is the size of problem. ($\Leftarrow$) From the definition of kernalization, we can find the kernel in $O(\text{poly}(|x| + k))$ time and then the solution in time bounded by $O(f(|x| + k)) \leq O(f(g(k)))$ which has no factor of n and hence, is an FPT.

($\Rightarrow$) Suppose $\mathcal{A}$ solves instance (x, k) of L in time $O(f(k) \cdot |x|^c)$.

Algorithm 5.2. Run $\mathcal{A}$ on (x, k) for $|x|^{c+1}$ steps

- If $\mathcal{A}$ stops and returns an answer, return a trivial Yes or No instance
- Else: return (x, k)

It is clear that Yes $\Leftarrow$ Yes. What about the “ $|x'| + k' \leq g(k)$ for some computable g ”?

Well, as $\mathcal{A}$ is an FPT with runtime $O(f(x) \cdot |x|^c)$, the fact the algorithm has not terminated tells us:

$$\begin{aligned} |x|^{c+1} &= O(f(k) |x|^c) \\ \Rightarrow |x|^{c+1} &\leq c_2 f(k) |x|^c \\ \Rightarrow |x| &\leq c_2 f(k) \\ \Rightarrow |x| &= O(f(k)) \end{aligned}$$

Thus, $|x| + k = O(f(k) + k)$.

Thus, we have a kernel of size $O(f(k) + k)$. ■

Notice, the FPT doesn't give a particularly useful kernalization. It is just a nice statement to say.

5.1. Planar Independent Set

Definition: Independent Set

Input: Given a graph G and $k \in \mathbb{N}$

Question: Does there exist $S \subseteq V$ such that $|S| \geq k$ such that $u, v \in S \Leftrightarrow uv \notin E$?

While in the classical setting, this and vertex cover are twins; The same doesn't track here. If we reduce to vertex cover, we have a $O^*(2^{n-k})$ algorithm which is not FPT.

Sadly, there is no FPT using the standard parametrization as the problem is $W[1]$ hard with respect to k .

However, if we restrict to the planar case, we can however have a kernel.

Definition: Planar Independent Set

Input: Given a planar graph G and $k \in \mathbb{N}$

Question: Does there exist $S \subseteq V$ such that $|S| \geq k$ such that $u, v \in S \Leftrightarrow uv \notin E$?

While this NP Hard (even on bounded degree), we now have a linear kernel (and hence, FPT) on the standard parametrization: if $|V| \geq 4k$ then by 4 color theorem, return **Yes**. Otherwise, return (G, k) .

This is sometimes called a *cheat* kernel as we don't really find out much about the problem.

6. Feedback Vertex Set

Definition: Feedback Vertex Set

Input: Given an undirected graph G on n vertices, $k \in \mathbb{N}$.

Parameter: k

Question: Does G contain a set S of size $\leq k$ such that $G - S$ has no cycles.

We call $S \subseteq V(G)$ is a feedback vertex set of G if and only if $G - S$ is acyclic.

The idea is similar to vertex cover. We first delete all degree 0 and 1 vertices.

Now that we have a graph with degree atleast 2, there must either be a

TODO.

Theorem 6.1. If $n < (2 \log_2 n)^k$ then:

$$(2 \log_2 n)^k < (4k \log_2 n)^k$$

TODO. *hain*

Proof.

TODO. *hain*

■

7. Feedback Arc Set

Definition: Feedback Arc Set in Tournament

Input: Given a tournament T on n vertices, $k \in \mathbb{N}$

Parameter: k

Question: Does T contain an arc set F of size $\leq k$, such that $T - F$ has no directed cycle?

Such a F is called a feedback arc set of tournament T .

We could begin with getting rid of all the clear winners (out-degree 0) and clear losers (in-degree 0).

Now the tournament is guaranteed to have a cycle, we use the following claim:

Claim 7.1. An arc (and vertex) in a tournament is part of directed cycle if and only if it is part of a directed triangle.

Proof. FTSOC let the arc uv be part of a the shortest cycle c which is not a triangle.

TODO.

■

Similar to vertex cover, we would like to sort of have a rule to deal with arc's which are part of large number of triangle (k). But we can't delete it as otherwise the instance is no longer a tournament. So what do we do?

Definition

For a subset F of arcs of T , let $T \oplus F$ be the tournament obtained from T by reversing the arcs in F

Claim 7.2. If $T \oplus F$ is acyclic then F is a feedback set of T

Claim 7.3. If F is an inclusion minimal feedback arc set of T then $T \oplus F$ is acyclic.

TODO. *proof is hw*

Corollary 7.4. If (T, k) is an Yes instance of FAST $\iff$ there is a set F of arcs of T , $|F| \leq k$, such that $T \oplus F$ is acyclic.

These gives us clear reduction rules (which are very similar to vertex cover):

- **Reduction Rule 1:** If T has vertex v which is not part of any triangle in T , return $(T - v, k)$
- **Reduction Rule 2:** If T has an arc e that is part of $\geq k + 1$ triangles, then return $(T \oplus \{e\}, k - 1)$

TODO. *Prove Validity of rule 2!*

But why is this a valid kernel?

Proof. If both the reduction rules fail and we are in a non-trivial instance, notice that every arc is part of atmost k triangles and every vertex is part of a triangle.

Hence, each arc covers atmost k triangles. Thus, reversing an edge can 'destroy' atmost k triangles which cover $k + 2$ vertices.

Thus, if the instance has $> k(k + 1)$ vertices, we can return a No instance. ■

This gives us a $k^2 + 2k + k = O(k^2)$ kernel.

Idea

The usual idea in coming up with kernalization algorithms is to come up with reduction rules based on ‘simple’ cases.

The way we come up with these rules is to find useful structures we can discover in polynomial time.

Some common structures are:

- Something that is definitely part of the solution
- Something that is definitely not part of optimal solution
- Something which we can replace with a smaller/simpler thing

Note, this same idea gives us a $O^*(3^k)$ FPT algorithm by branching on the triangles.

$O^*(2^{O(\sqrt{k} \log(k))})$ is the best known bound for FAST by Alon, Lokshtanov and Saurabh in 2009.

For FVS, the easy bound is $O^*(3^k)$. The iterated compression method gives $O^*(2^k)$ and the current best is $O^*(1.618^k)$ by Kumar¹ and Lokshtanov in 2016.

¹Another CMI person!